

The problem:

Not all state should

persist equally

Introduction: Where you are now

Modules 1-2 recap: You learned

- 1 Session state is a dictionary accessible via `session.state`
- 2 `output_key` automatically saves agent responses to state
- 3 `{var}` templating injects state values into instructions

You can now:

```
Python
from google.adk.agents import LlmAgent

# Save data with output_key
agent = LlmAgent(
    instruction="Extract the main topic",
    output_key="topic"
)

# Use templating to inject state values
agent = LlmAgent(
    instruction="Provide an overview of
{topic}"
)
```

The next question

How long does state persist?

```
Python
# All are written to state,
# but how long do they live?

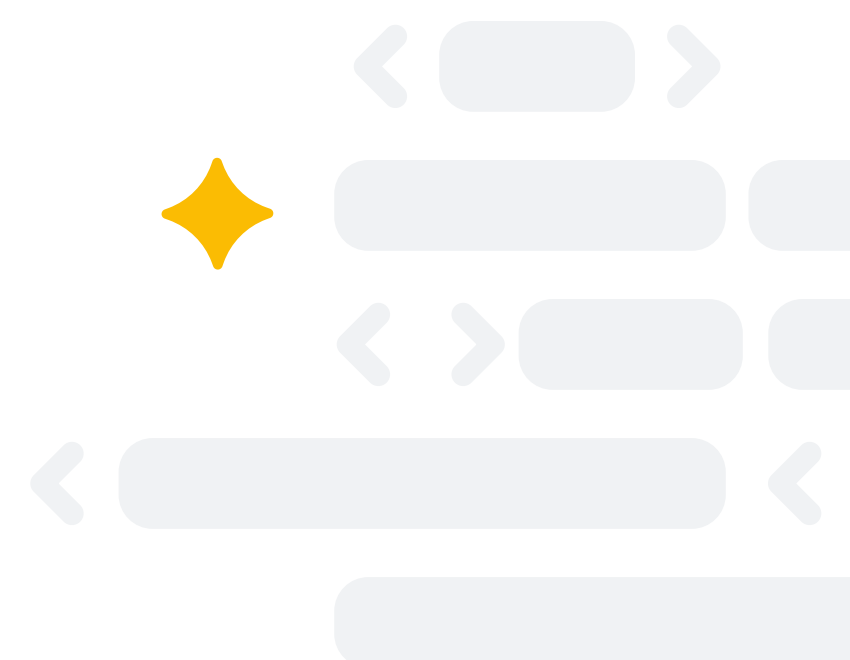
session.state["processing_step"] =
"validation"

session.state["user_language"] =
"Spanish"

session.state["api_endpoint"] =
"https://api.example.com"
```

The answer

State namespaces control persistence scope.



The problem:

Not all state should persist equally

Consider different types of data an agent needs:

Python

```
# These have VERY different lifetime requirements:
state["current_step"] = "validating"      # Only need right now
state["conversation_topic"] = "refunds"    # Need this conversation only
state["user_theme"] = "dark"              # Need across all conversations
state["api_url"] = "https://api.com"      # Global for all users
```

Problems without namespaces

- ✗ Temporary data persists forever
→ Memory leak over time
- ✗ User preferences lost when session ends
→ User must re-configure every time
- ✗ Global config stored per-session
→ Wasteful duplication
- ✗ Conversation data bleeds across sessions
→ Privacy and context issues

What you need

Different persistence scopes for different data types.

